Setup, Installation and Configuration

By : Varun Wilfred

CHAPTER 1

INTRODUCTION

1.1 Introduction

Modern organizations face an increasing number of cyber threats targeting publicly accessible systems and services. Traditional security mechanisms such as firewalls and intrusion detection systems primarily focus on preventing attacks, but they often provide limited insight into the techniques and behavior of attackers after an intrusion attempt begins. Honeypots address this challenge by acting as decoy systems that intentionally attract malicious actors while safely monitoring and recording their activities.

This project presents the implementation of a T-Pot Honeypot Lab, a virtualized cybersecurity environment designed to capture, monitor, and analyze attacker behavior in a controlled setting. The lab was deployed using Ubuntu Server as the host operating system and the T-Pot Community Edition platform, which integrates multiple honeypots and security monitoring tools into a single solution. The environment was configured inside a virtual machine, allowing safe experimentation without affecting production systems.

The completed environment includes several specialized honeypots capable of emulating different network services, while the integrated Elastic Stack collects, stores, and visualizes attack data through Kibana dashboards. By generating controlled attacks from a Kali Linux virtual machine, the project demonstrates how reconnaissance activities, login attempts, command execution, and other malicious interactions can be captured and analyzed in real time.

The primary objective of this report is to document the complete setup, installation, and configuration process of the T-Pot Honeypot Lab. It serves as a technical implementation guide describing the hardware requirements, software components, deployment procedure, configuration steps, and verification process required to build a fully functional honeypot environment.

CHAPTER 2

OBJECTIVES

2.1 Primary Objective

The primary objective of this project is to design, deploy, and configure a fully functional T-Pot Honeypot Lab capable of monitoring, recording, and analyzing malicious activities within a secure virtual environment. The project aims to provide hands-on experience in deploying modern honeypot technologies while understanding how attackers interact with exposed network services.

2.2 Specific Objectives

The specific objectives of this project are:

- To install and configure Ubuntu Server as the operating system for hosting the honeypot platform.
- To deploy the T-Pot Community Edition using Docker containers.
- To configure the virtual machine with appropriate hardware resources and bridged network connectivity.
- To verify the successful deployment of multiple honeypot services, including Cowrie, Honeytrap, Dionaea, ConPot, Tanner, and supporting monitoring components.
- To validate the functionality of the environment by generating controlled attack traffic from a Kali Linux virtual machine.
- To ensure that attack events are successfully collected, indexed, and visualized using the Elastic Stack and Kibana dashboards.
- To create a secure and isolated cybersecurity laboratory suitable for studying attacker behavior without exposing production systems.

2.3 Expected Outcomes

Upon successful completion of this project, the following outcomes were expected:

- A fully operational T-Pot Honeypot Lab running within a virtualized environment.
- Successful deployment of multiple honeypot services capable of attracting and logging attacker activity.
- Real-time monitoring and visualization of collected security events through Kibana dashboards.
- A stable platform for conducting future cybersecurity research, attack simulations, and threat analysis exercises.

CHAPTER 3

PROJECT OVERVIEW

3.1 Overview

The T-Pot Honeypot Lab is a cybersecurity project developed to create a secure environment for observing, capturing, and analyzing malicious network activities. Instead of protecting a production system, the platform intentionally exposes simulated services that attract attackers while safely recording every interaction. This enables security professionals and researchers to study attack techniques, identify common reconnaissance methods, and improve defensive strategies without exposing real assets to risk.

T-Pot Community Edition was selected because it provides an integrated honeypot platform containing multiple specialized honeypots and security monitoring tools within a single deployment. The platform combines Docker container technology with the Elastic Stack to deliver centralized log collection, real-time visualization, and attack analysis through an intuitive web interface.

Unlike a standalone honeypot, T-Pot supports several network services simultaneously, allowing different types of attacks to be captured within the same environment. During this project, the platform successfully detected network reconnaissance, Telnet interactions, service enumeration, and command execution performed from a Kali Linux virtual machine. All captured events were forwarded to Elasticsearch and visualized through Kibana dashboards for further analysis.

The entire laboratory was implemented inside a virtual machine running Ubuntu Server, while Kali Linux served as the attacker machine for generating controlled attack traffic. This isolated architecture ensured that all testing activities remained within the virtual environment, providing a safe platform for practical cybersecurity experimentation and future threat analysis.

3.2 Project Workflow

The implementation of the project followed the workflow below:

1. Configure the virtual machine with the required hardware resources.
2. Install Ubuntu Server and perform initial system configuration.
3. Install Docker and deploy the T-Pot Community Edition.
4. Verify the successful deployment of all honeypot services.
5. Access the T-Pot web interface and Kibana dashboards.

6. Generate controlled attacks from the Kali Linux virtual machine.
7. Capture, store, and visualize attack events using the Elastic Stack.
8. Validate that the honeypot environment successfully recorded attacker activities.

CHAPTER 4

LAB ARCHITECTURE

4.1 Overview

The T-Pot Honeypot Lab was designed as an isolated virtual environment to safely monitor and analyze malicious network activities. The architecture consists of two virtual machines hosted on a MacBook Air M2 using UTM virtualization software. Ubuntu Server 24.04 LTS was configured as the T-Pot server, while Kali Linux was used as the attacker machine for generating controlled attack traffic.

The T-Pot server hosts multiple Docker containers, each running a dedicated honeypot or monitoring component. These services collect attacker interactions and forward the generated logs to the Elastic Stack for centralized storage, indexing, and visualization. Kibana provides a web-based dashboard that enables real-time monitoring of attack events, while the integrated Attack Map offers a graphical representation of detected attacks.

During the project, Kali Linux performed reconnaissance, authentication attempts, and simulated attacker activities against the exposed honeypot services. The resulting events were captured by T-Pot and stored within Elasticsearch, allowing detailed analysis through Kibana dashboards and Discover.

4.2 Lab Components

The laboratory environment consisted of the following components:

ComponentPurpose
MacBook Air M2Physical host system
UTMVirtualization platform

Ubuntu Server 24.04 LTSOperating system hosting T-Pot
T-Pot Community Edition 24.04.1Multi-honeypot platform
DockerContainer runtime for T-Pot services
ElasticsearchLog storage and indexing
KibanaDashboard and log visualization
CowrieSSH and Telnet honeypot
HoneytrapGeneric network honeypot
DionaeaMalware collection honeypot
ConPotIndustrial Control System (ICS) honeypot
TannerWeb application honeypot
Kali LinuxAttacker machine used for controlled testing

4.3 Network Architecture

Both virtual machines were connected using a bridged network configuration, allowing the Kali Linux attacker machine to communicate directly with the T-Pot server. This configuration enabled realistic attack simulations while keeping all activities contained within the virtual laboratory environment. All generated traffic remained isolated from production systems, ensuring safe testing and analysis.

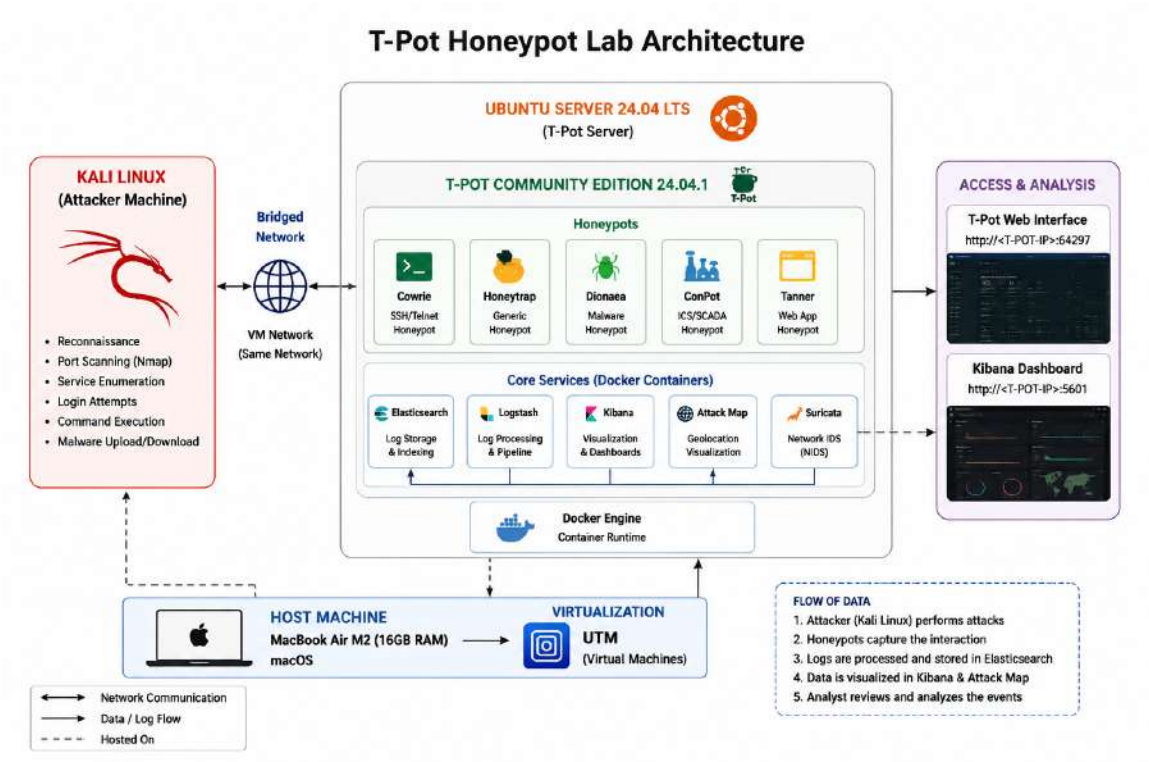


Figure 4.1: T-Pot Honeypot Lab Architecture

CHAPTER 5

HARDWARE AND SOFTWARE REQUIREMENTS

5.1 Overview

Before deploying the T-Pot Honeypot Lab, the necessary hardware and software requirements were identified to ensure stable performance of the virtual environment. The project was implemented on a MacBook Air M2 using UTM virtualization software. Ubuntu Server 24.04 LTS was selected as the operating system for hosting the T-Pot Community Edition, while Kali Linux was used as the attacker machine to generate controlled cyberattack simulations. Docker was used as the containerization platform for deploying the various honeypot services integrated within T-Pot.

5.2 Hardware Requirements

The following hardware specifications were used during the implementation of the project:

Component	Specification
Host System	MacBook Air M2
Processor	Apple M2 Chip
Memory	16 GB RAM
Storage	256 GB SSD
Virtualization Software	UTM

The T-Pot virtual machine was allocated the following resources:

Resource	Conf
CPU	4 Virtual Cores
RAM	8 GB
Storage	100 GB
Network	Bridged Adapter

5.3 Software Requirements

The software components used throughout the project are listed below:

Software	Purpose
Ubuntu Server 24.04 LTS	Host operating system
T-Pot Community Edition 24.04.1	Multi-honeypot platform
Docker	Container runtime
Elasticsearch	Log storage and indexing
Kibana	Data visualization and analysis
Kali Linux	Attacker virtual machine
Nmap	Network reconnaissance
Telnet	Authentication testing
Nikto	Web vulnerability scanning

5.4 Development Environment

The implementation was carried out in an isolated virtual laboratory to ensure that all attack simulations remained within a controlled environment. Kali Linux generated reconnaissance scans and authentication attempts against the T-Pot server, while the integrated honeypots recorded attacker interactions. The collected logs were processed by Elasticsearch and visualized through Kibana dashboards, enabling real-time monitoring and analysis of simulated cyberattacks.

The selected hardware configuration provided sufficient computational resources to run multiple Docker containers simultaneously while maintaining stable performance throughout the deployment, testing, and analysis phases of the project.

CHAPTER 6

TOOLS AND TECHNOLOGIES

6.1 Overview

The successful implementation of the T-Pot Honeypot Lab required the integration of multiple software tools and technologies. Each component was selected to perform a specific function, including virtualization, operating system management, containerization, attack simulation, log collection, and security event visualization. Together, these technologies created a complete environment for deploying and analyzing a modern multi-honeypot platform.

6.2 Technologies Used

6.2.1 UTM

UTM was used as the virtualization platform to host the Ubuntu Server and Kali Linux virtual machines. It provided hardware virtualization on the Apple M2 architecture while allowing both operating systems to communicate through a bridged network configuration.

6.2.2 Ubuntu Server 24.04 LTS

Ubuntu Server served as the operating system for hosting the T-Pot Community Edition. It provided a stable Linux environment capable of supporting Docker containers and various security monitoring services.

6.2.3 Docker

Docker was used as the containerization platform for deploying the different honeypot services included in T-Pot. Each honeypot and monitoring component operated within its own isolated container, simplifying deployment, management, and maintenance.

6.2.4 T-Pot Community Edition

T-Pot Community Edition is an open-source multi-honeypot platform that combines numerous honeypot services with centralized monitoring and visualization tools. It automatically deploys multiple Docker containers that emulate vulnerable services while collecting and analyzing attacker activities.

6.2.5 Elasticsearch

Elasticsearch served as the centralized data storage and indexing engine. Security events generated by the honeypots were processed and stored efficiently, enabling fast searching and analysis of collected logs.

6.2.6 Kibana

Kibana provided a web-based interface for visualizing and analyzing the collected security events. Interactive dashboards, graphs, attack statistics, and search capabilities enabled real-time monitoring of attacker behavior.

6.2.7 Kali Linux

Kali Linux was used as the attacker machine throughout the project. It generated controlled cyberattacks against the T-Pot server for testing the functionality of the deployed honeypots.

6.2.8 Nmap

Nmap was used to perform network reconnaissance and service enumeration against the T-Pot server. The generated traffic allowed the honeypots to capture reconnaissance activities and record them for analysis.

6.2.9 Telnet

Telnet was used to interact with the Cowrie honeypot. Successful login attempts into the simulated environment demonstrated how attacker activities could be safely monitored and recorded without compromising a real system.

6.2.10 Nikto

Nikto was used to perform web server scanning against the honeypot environment. The generated HTTP requests allowed the platform to record web-based reconnaissance and scanning activities for later analysis.

6.3 Technology Integration

The combination of these technologies created a complete cybersecurity laboratory capable of deploying multiple honeypots, capturing attacker interactions, processing security logs, and presenting the collected information through interactive dashboards. The integration of virtualization, Docker containers, Elastic Stack, and offensive security tools enabled the project to simulate realistic cyberattack scenarios while maintaining a safe and isolated testing environment.

CHAPTER 7

VIRTUAL MACHINE CONFIGURATION

7.1 Overview

Virtualization plays an important role in cybersecurity by enabling secure and isolated environments for testing and analysis. In this project, UTM virtualization software was used to create separate virtual machines for the T-Pot Honeypot server and the Kali Linux attacker system. This approach ensured that all simulated cyberattacks remained confined within the virtual laboratory without affecting the host operating system or external networks.

The virtual machine hosting T-Pot was configured with sufficient hardware resources to support multiple Docker containers running simultaneously. A bridged network adapter was selected to allow direct communication between the attacker machine and the honeypot environment.

7.2 T-Pot Virtual Machine Configuration

The Ubuntu Server virtual machine was configured with the following specifications before installing T-Pot Community Edition.

Configuration	Value
Operating System	Ubuntu Server 24.04 LTS
CPU Allocation	4 Virtual CPU Cores
Memory	8 GB RAM
Storage	100 GB Virtual Disk
Network Mode	Bridged Network
Virtualization Platform	UTM

The selected configuration provided sufficient resources for Docker containers, Elasticsearch, Kibana, and the integrated honeypot services to operate efficiently during testing and analysis.

7.3 Kali Linux Virtual Machine

A separate Kali Linux virtual machine was deployed to simulate attacker activities. The system was configured to communicate directly with the T-Pot server through the bridged network, enabling realistic attack simulations while maintaining an isolated testing environment.

The Kali Linux virtual machine was primarily used to perform:

* Network reconnaissance using Nmap.
* Service detection and enumeration.
* Telnet authentication testing.
* Web server scanning using Nikto.
* Controlled interaction with the Cowrie honeypot.

7.4 Network Configuration

Both virtual machines were connected using a bridged network adapter, allowing them to obtain IP addresses from the local network and communicate directly with one another. This configuration closely resembles a real-world deployment and enables accurate simulation of attacker behavior against exposed services.

The bridged networking approach also ensured that T-Pot could receive and record all incoming traffic generated by the Kali Linux attacker machine while remaining isolated from production systems.

7.5 Verification

After completing the virtual machine configuration, both Ubuntu Server and Kali Linux were successfully deployed and verified. Network connectivity between the two systems was confirmed before proceeding with Docker installation and T-Pot deployment, ensuring that the environment was fully prepared for the implementation of the honeypot platform.

CHAPTER 8

UBUNTU SERVER INSTALLATION

8.1 Overview

Ubuntu Server 24.04 LTS was selected as the operating system for hosting the T-Pot Honeypot Lab. The server edition was chosen because it provides a lightweight, stable, and secure environment optimized for running containerized applications and network services.

The operating system was installed on a virtual machine created using UTM with 4 virtual CPU cores, 8 GB RAM, and 64 GB of storage. After the installation was completed, the system was updated and prepared for the deployment of T-Pot Community Edition.

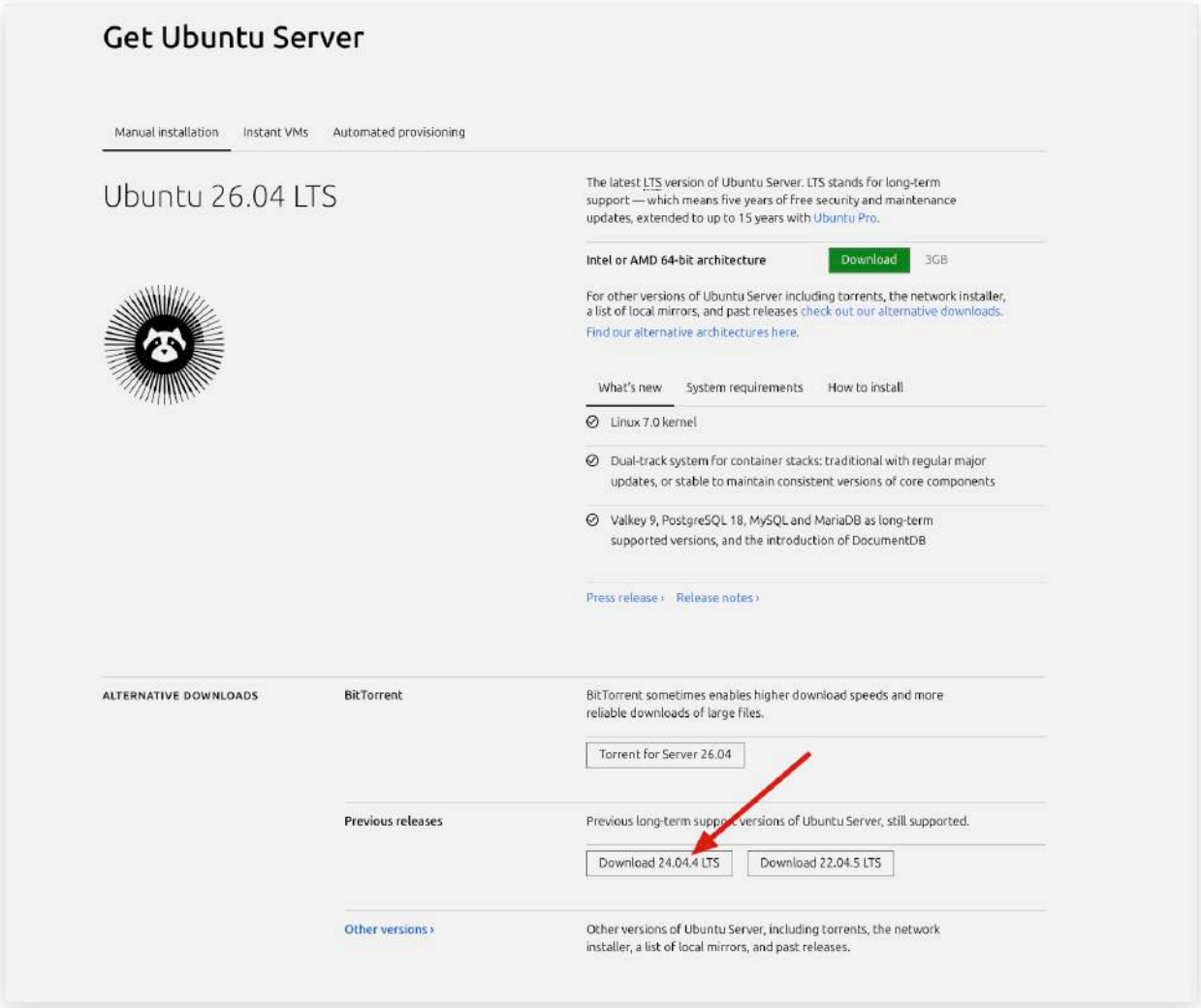


Figure 8.1: Ubuntu Server 24.04 LTS Official Release

8.2 Summary

With Ubuntu Server successfully installed and configured, the environment was ready for deploying the T-Pot Community Edition using the official installation script. The following chapter describes the installation and configuration process of the T-Pot platform.

CHAPTER 9

T-POT INSTALLATION

9.1 Overview

Following the successful installation of Ubuntu Server 24.04 LTS, the T-Pot Community Edition was deployed using the official installation script provided by the T-Pot developers. The installer automatically prepared the server environment by installing the required dependencies, configuring Docker, downloading the necessary container images, and deploying the integrated honeypot services.

This automated installation process simplified the deployment of the multi-honeypot platform while ensuring that all components were configured correctly.

Visit : Github

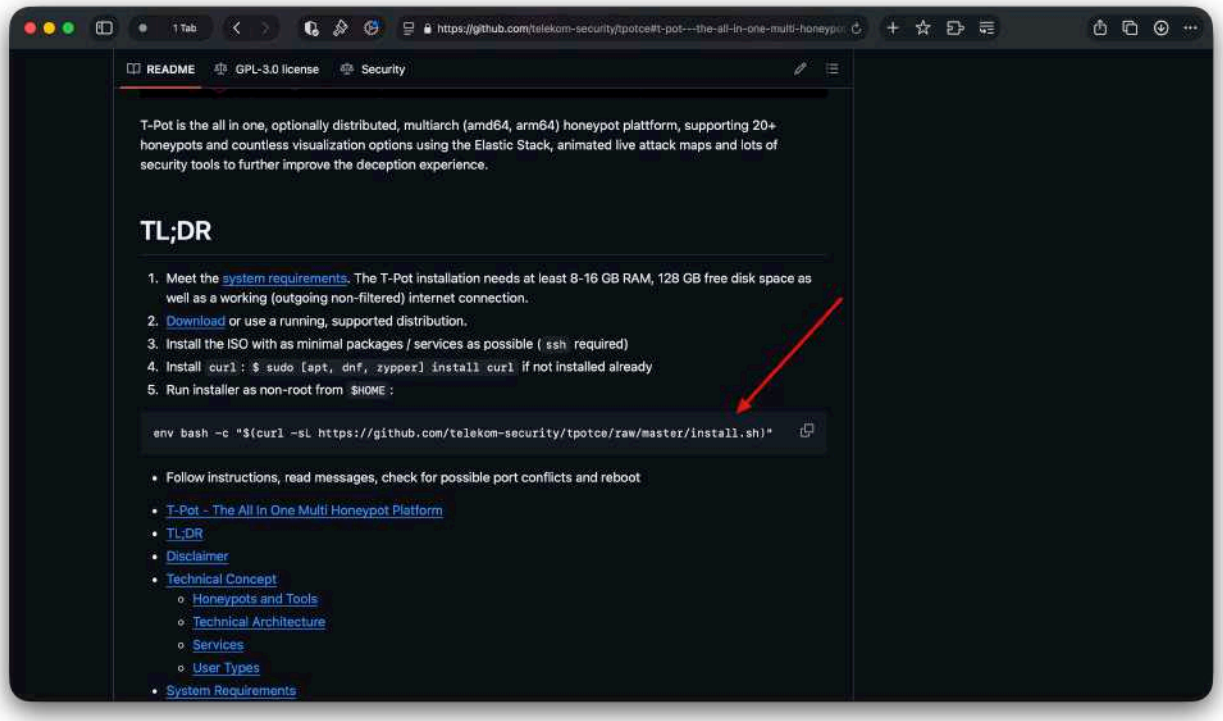


Figure 9.1: Official T-Pot Installation Command

9.2 Installation Process

The installation script was executed from the Ubuntu terminal using an internet connection. During the installation, the script verified the operating system, installed the required software packages, configured Docker services, and downloaded the necessary container images.

After the installation process began, the **Hive** deployment profile was selected to enable the complete T-Pot environment with multiple integrated honeypots and centralized monitoring capabilities.

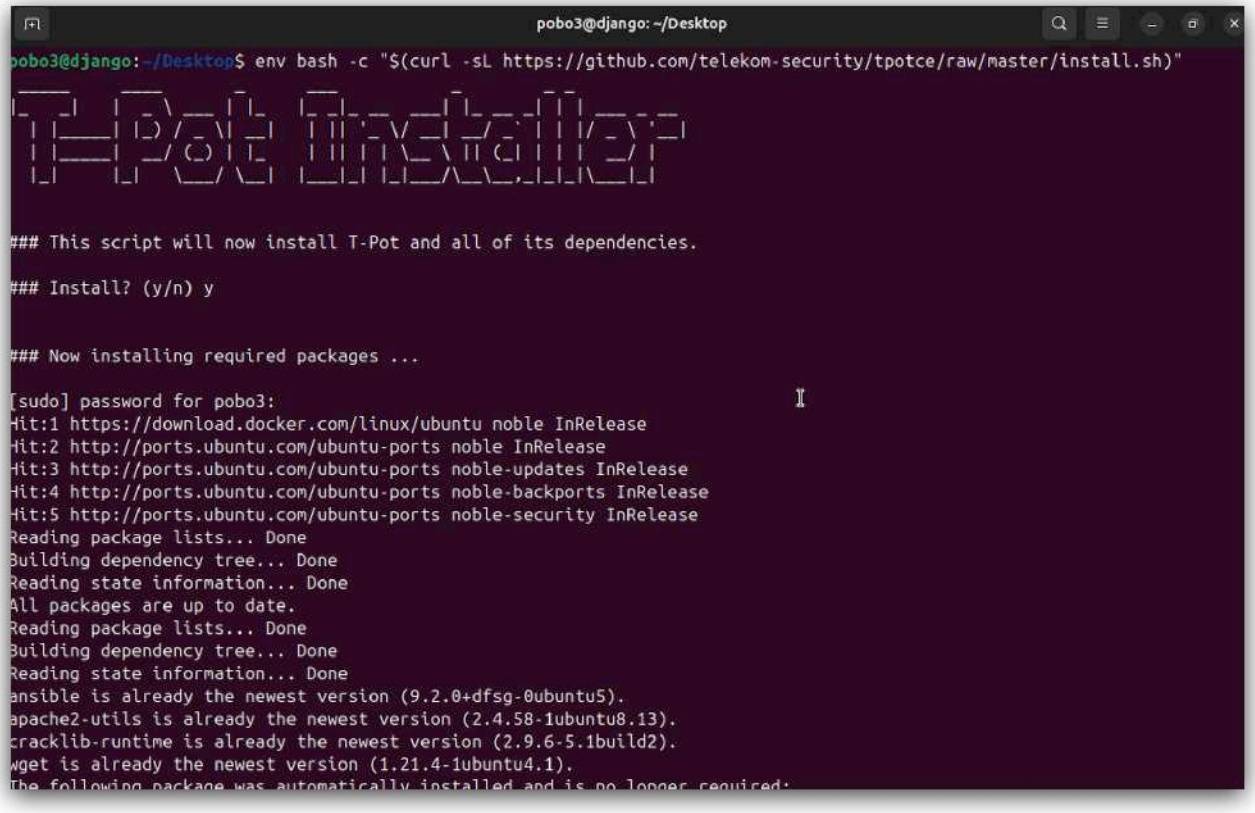
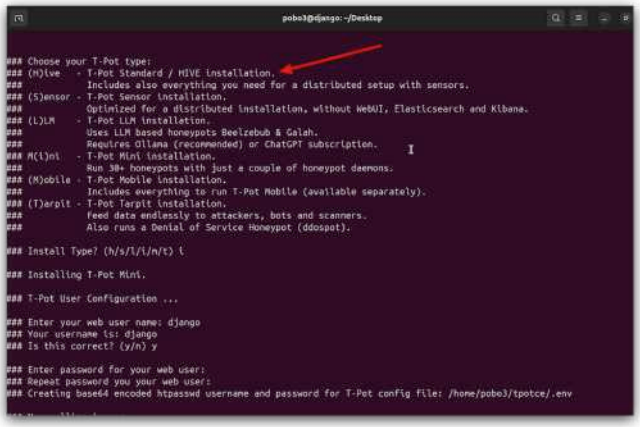


Figure 9.2: T-Pot Installation Process

Choose Hive as the Standard process :



9.3 Installation Completion

Once all required components had been downloaded and configured, the installation completed successfully. The installer displayed a confirmation message indicating that the deployment was complete and instructed the user to restart the system before accessing the T-Pot web interface.

After rebooting, the platform automatically initialized the Docker containers and prepared all integrated services for operation.

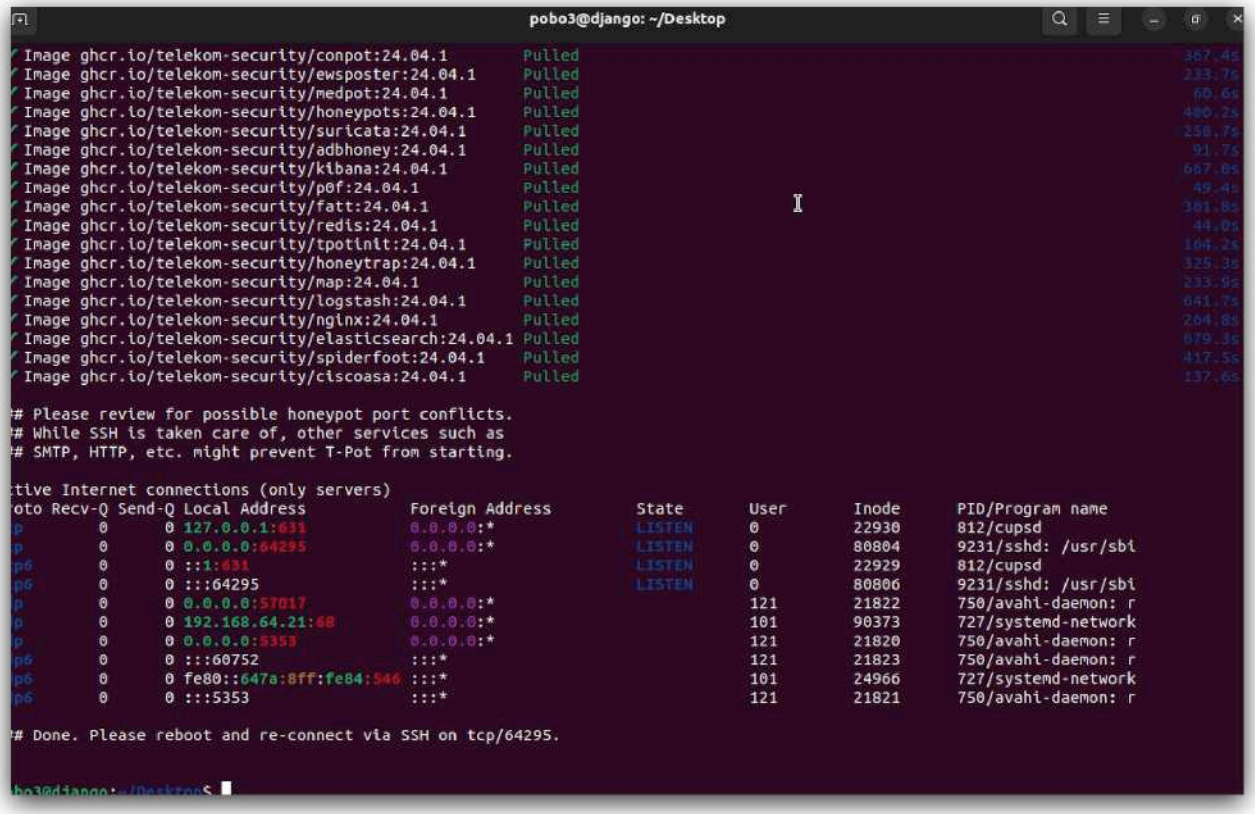


Figure 9.3: Successful Installation of T-Pot

9.4 Summary

The successful deployment of T-Pot Community Edition established the foundation of the Honeypot Lab. All required services, Docker containers, and monitoring components were installed automatically through the official installation script, preparing the environment for configuration and verification in the following chapter.

CHAPTER 10

INITIAL CONFIGURATION

10.1 Overview

Following the successful installation of T-Pot Community Edition, the server was restarted to initialize all required services and Docker containers. The initial configuration phase focused on verifying that the T-Pot service was running correctly, confirming the status of the deployed containers, and ensuring that the web interface and monitoring dashboards were accessible before beginning attack simulations.

10.2 Verifying the T-Pot Service

The operational status of the T-Pot platform was verified using the Linux system service manager. The command confirmed that the T-Pot service was active and running successfully, indicating that the platform had started correctly after the system reboot.

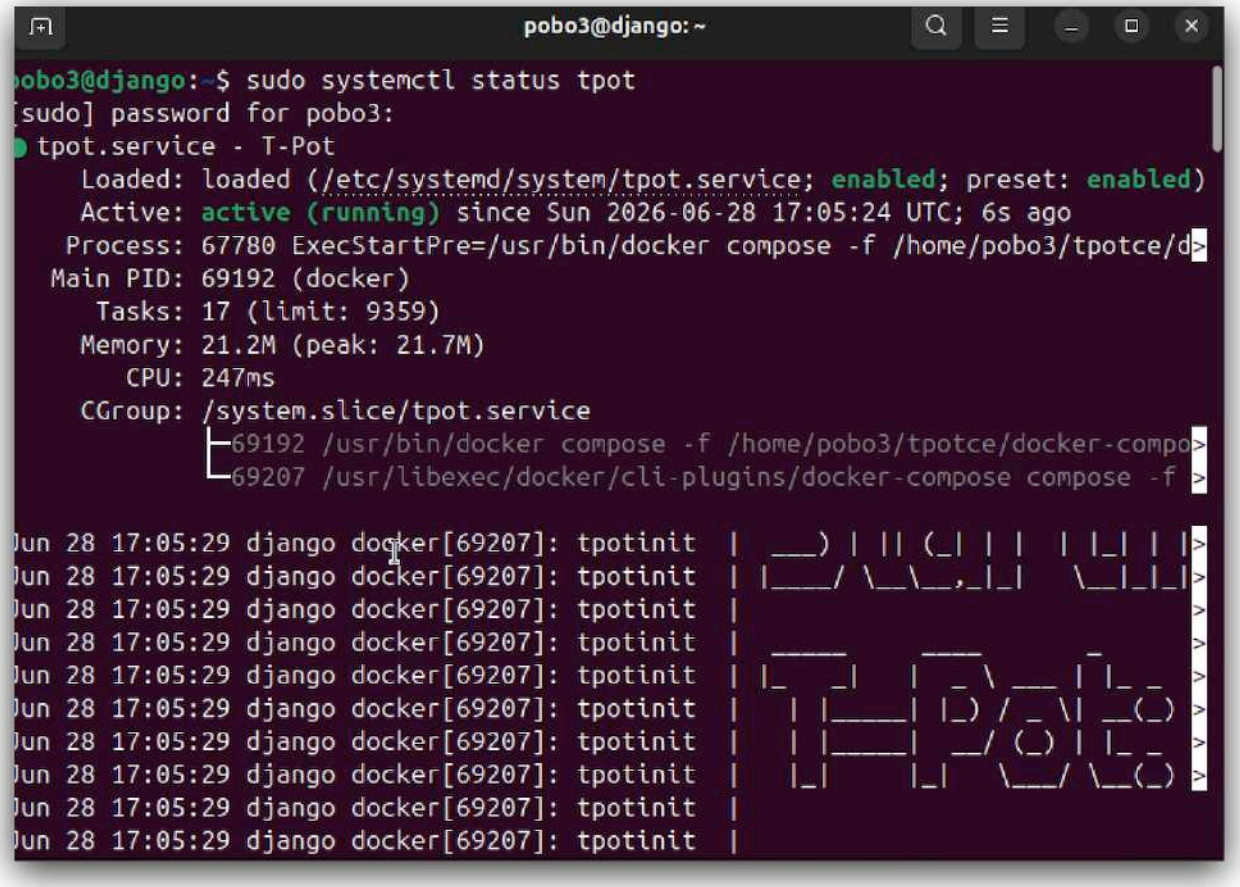


Figure 10.1: T-Pot Service Status (`sudo systemctl status tpot`)

10.3 Verifying Docker Containers

After confirming that the T-Pot service was active, the running Docker containers were verified to ensure that the integrated honeypots and supporting services had been deployed successfully. The output confirmed that essential components such as Elasticsearch, Kibana, Cowrie, Honeytrap, Dionaea, ConPot, and other services were running correctly.

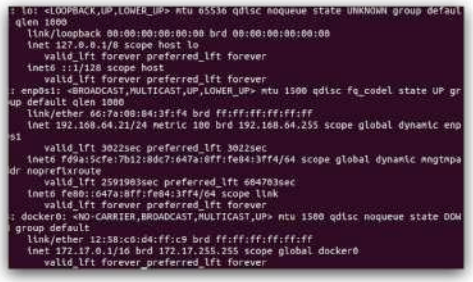


Figure 10.2: Running Docker Containers (`sudo docker ps`)

10.4 Accessing the T-Pot Web Interface

The Ubuntu Server IP address was used to access the T-Pot web interface through a web browser. The landing page successfully displayed the available applications, including Kibana, SpiderFoot, the Attack Map, and other integrated services.

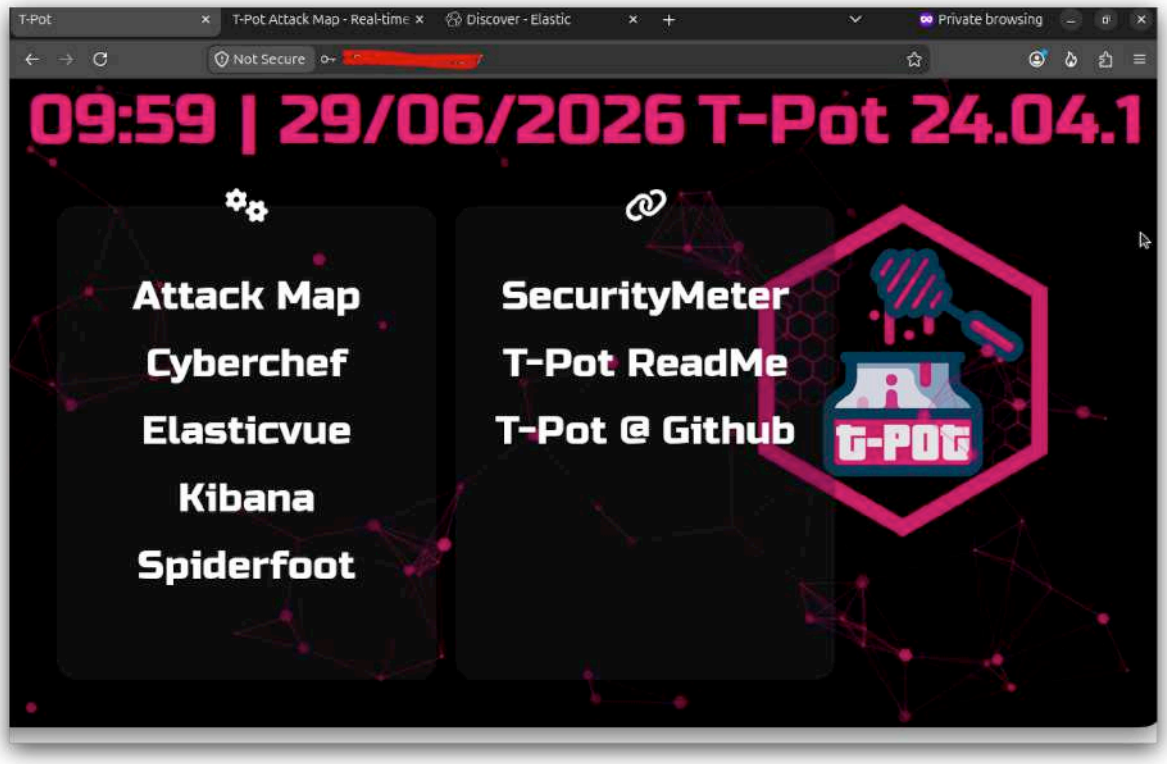


Figure 10.3: T-Pot Landing Page

10.5 Accessing the Kibana Dashboard

The Kibana dashboard was opened through the T-Pot web interface to verify communication between the honeypot services and the Elastic Stack. The successful loading of the dashboard confirmed that security events could be collected and visualized within the platform.

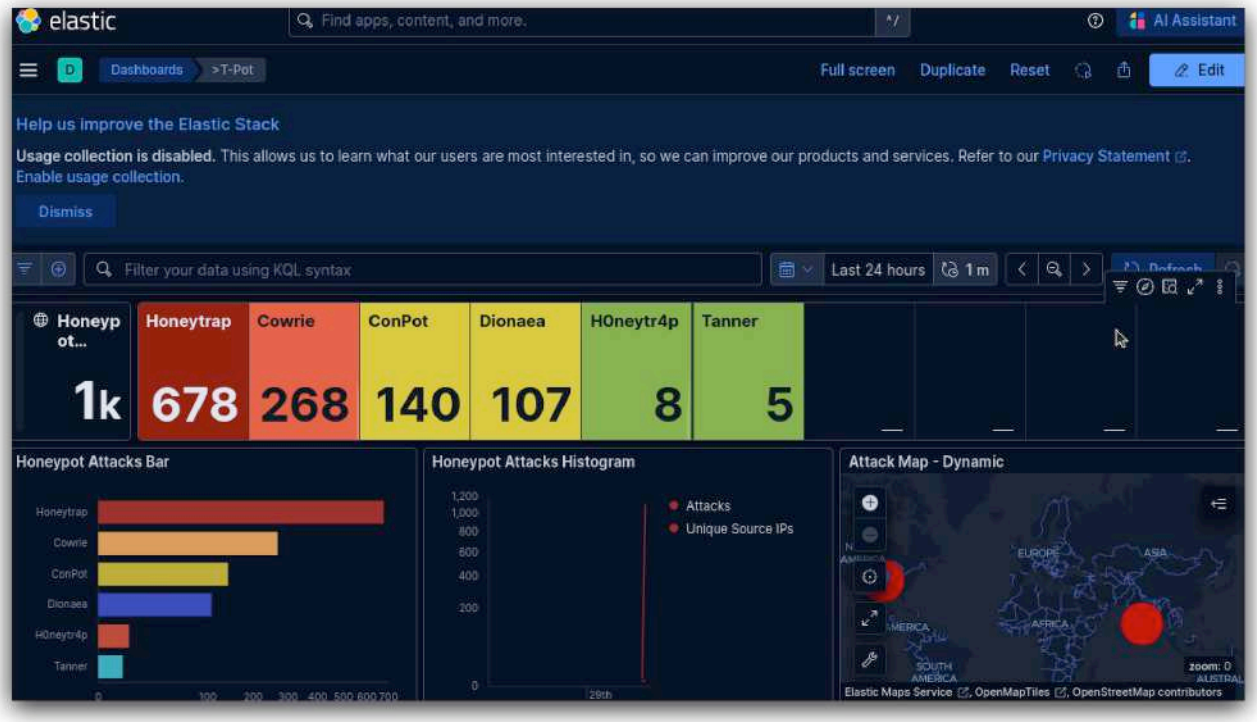


Figure 10.4: Kibana Dashboard

10.6 Summary

The initial configuration process verified that the T-Pot service, Docker containers, web interface, and Kibana dashboard were functioning correctly. With all major components successfully initialized, the honeypot environment was fully prepared for attack simulation and threat analysis.